

big data in finance

Hiba Majdoubi

2025-04-23

```
#daily data
X25_book_to_market_Daily <- read_csv("C:/Users/hibam/OneDrive/Documents/Master Actuariat/Big data in Finance/2025-04-23/X25_book_to_market_Daily.csv")

## New names:
## * ` ` -> `...1`

## Warning: One or more parsing issues, call `problems()` on your data frame for details,
## e.g.:
##   dat <- vroom(...)
##   problems(dat)

## Rows: 103611 Columns: 26
## -- Column specification -----
## Delimiter: ","
## chr  (1): ...1
## dbl (25): SMALL LoBM, ME1 BM2, ME1 BM3, ME1 BM4, SMALL HiBM, ME2 BM1, ME2 BM...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.

X100_book_to_market_Daily <- read_csv("C:/Users/hibam/OneDrive/Documents/Master Actuariat/Big data in Finance/2025-04-23/X100_book_to_market_Daily.csv")

## New names:
## * ` ` -> `...1`

## Warning: One or more parsing issues, call `problems()` on your data frame for details,
## e.g.:
##   dat <- vroom(...)
##   problems(dat)

## Rows: 103611 Columns: 101
## -- Column specification -----
## Delimiter: ","
## chr  (1): ...1
## dbl (100): SMALL LoBM, ME1 BM2, ME1 BM3, ME1 BM4, ME1 BM5, ME1 BM6, ME1 BM7,...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
X10_Industry_Portfolios_Daily <- read_csv("C:/Users/hibam/OneDrive/Documents/Master Actuariat/Big data/
```

```
## New names:
```

```
## * `` -> `...1`
```

```
## Warning: One or more parsing issues, call `problems()` on your data frame for details,
```

```
## e.g.:
```

```
##   dat <- vroom(...)
```

```
##   problems(dat)
```

```
## Rows: 51805 Columns: 11
```

```
## -- Column specification -----
```

```
## Delimiter: ","
```

```
## chr  (1): ...1
```

```
## dbl (10): NoDur, Durbl, Manuf, Enrgy, HiTec, Telcm, Shops, Hlth, Utils, Other
```

```
##
```

```
## i Use `spec()` to retrieve the full column specification for this data.
```

```
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
X48_Industry_Portfolios_Daily <- read_csv("C:/Users/hibam/OneDrive/Documents/Master Actuariat/Big data/
```

```
## New names:
```

```
## * `` -> `...1`
```

```
## Warning: One or more parsing issues, call `problems()` on your data frame for details,
```

```
## e.g.:
```

```
##   dat <- vroom(...)
```

```
##   problems(dat)
```

```
## Rows: 51805 Columns: 49
```

```
## -- Column specification -----
```

```
## Delimiter: ","
```

```
## chr  (1): ...1
```

```
## dbl (48): Agric, Food, Soda, Beer, Smoke, Toys, Fun, Books, Hshld, Clths, Hl...
```

```
##
```

```
## i Use `spec()` to retrieve the full column specification for this data.
```

```
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
# Mensuel data
```

```
X25_book_to_market <- read_csv("C:/Users/hibam/OneDrive/Documents/Master Actuariat/Big data in Finance/
```

```
## New names:
```

```
## * `` -> `...1`
```

```
## Warning: One or more parsing issues, call `problems()` on your data frame for details,
```

```
## e.g.:
```

```
##   dat <- vroom(...)
```

```
##   problems(dat)
```

```

## Rows: 8797 Columns: 26
## -- Column specification -----
## Delimiter: ","
## chr (1): ...1
## dbl (25): SMALL LoBM, ME1 BM2, ME1 BM3, ME1 BM4, SMALL HiBM, ME2 BM1, ME2 BM...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.

X100_book_to_market <- read_csv("C:/Users/hibam/OneDrive/Documents/Master Actuariat/Big data in Finance

## New names:
## * `` -> `...1`

## Warning: One or more parsing issues, call `problems()` on your data frame for details,
## e.g.:
##   dat <- vroom(...)
##   problems(dat)

## Rows: 8797 Columns: 101
## -- Column specification -----
## Delimiter: ","
## chr (1): ...1
## dbl (100): SMALL LoBM, ME1 BM2, ME1 BM3, ME1 BM4, ME1 BM5, ME1 BM6, ME1 BM7,...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.

X10_Industry_Portfolios <- read_csv("C:/Users/hibam/OneDrive/Documents/Master Actuariat/Big data in Fin

## New names:
## * `` -> `...1`

## Warning: One or more parsing issues, call `problems()` on your data frame for details,
## e.g.:
##   dat <- vroom(...)
##   problems(dat)

## Rows: 5137 Columns: 11
## -- Column specification -----
## Delimiter: ","
## chr (11): ...1, NoDur, Durbl, Manuf, Enrgy, HiTec, Telcm, Shops, Hlth, Utils...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.

X48_Industry_Portfolios <- read_csv("C:/Users/hibam/OneDrive/Documents/Master Actuariat/Big data in Fin

## New names:
## * `` -> `...1`

```

```
## Warning: One or more parsing issues, call `problems()` on your data frame for details,
## e.g.:
##   dat <- vroom(...)
##   problems(dat)
```

```
## Rows: 5137 Columns: 49
## -- Column specification -----
## Delimiter: ","
## chr (49): ...1, Agric, Food, Soda, Beer, Smoke, Toys, Fun, Books, Hshld, Clt...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
# clean data, year format, NAs, value-weighted returns %
```

```
clean_dataframe <- function(df, output_path, date_format = "monthly") {
```

```
  df_clean <- df %>%
    rename(YearDate = 1) %>%
    mutate(
      Date = case_when(
        date_format == "monthly" ~ ymd(paste0(YearDate, "01")),
        date_format == "daily" ~ ymd(YearDate)
      ),
      across(-c(YearDate, Date), ~ as.numeric(.x)),
      across(-c(YearDate, Date), ~ na_if(.x, -99.99)),
      across(-c(YearDate, Date), ~ na_if(.x, -999)),
      across(-c(YearDate, Date), ~ .x / 100)
    ) %>%
    select(-YearDate) %>%
    filter(Date >= as.Date("1970-01-01") & Date <= as.Date("2024-12-31"))
```

```
# Export en CSV propre
```

```
write_csv(df_clean, file = output_path)
```

```
  return(df_clean)
```

```
}
```

```
D25_book_to_market <- clean_dataframe(X25_book_to_market_Daily, "vw_25_daily.csv", date_format = "daily")
```

```
## Warning: There were 2 warnings in `mutate()`.
## The first warning was:
## i In argument: `Date = case_when(...)`
## Caused by warning:
## ! All formats failed to parse. No formats found.
## i Run `dplyr::last_dplyr_warnings()` to see the 1 remaining warning.
```

```
D100_book_to_market <- clean_dataframe(X100_book_to_market_Daily, "vw_100_daily.csv", date_format = "daily")
```

```
## Warning: There were 2 warnings in `mutate()`.
## The first warning was:
```

```
## i In argument: `Date = case_when(...)`.  
## Caused by warning:  
## ! All formats failed to parse. No formats found.  
## i Run `dplyr::last_dplyr_warnings()` to see the 1 remaining warning.
```

```
D10_Industry_Portfolios <- clean_dataframe(X10_Industry_Portfolios_Daily, "vw_10_daily.csv", date_format = "m/d/yyyy")
```

```
## Warning: There were 2 warnings in `mutate()`.  
## The first warning was:  
## i In argument: `Date = case_when(...)`.  
## Caused by warning:  
## ! All formats failed to parse. No formats found.  
## i Run `dplyr::last_dplyr_warnings()` to see the 1 remaining warning.
```

```
D48_Industry_Portfolios <- clean_dataframe(X48_Industry_Portfolios_Daily, "vw_48_daily.csv", date_format = "m/d/yyyy")
```

```
## Warning: There were 2 warnings in `mutate()`.  
## The first warning was:  
## i In argument: `Date = case_when(...)`.  
## Caused by warning:  
## ! All formats failed to parse. No formats found.  
## i Run `dplyr::last_dplyr_warnings()` to see the 1 remaining warning.
```

```
M10_Industry_Portfolios <- clean_dataframe(X10_Industry_Portfolios, "vw_10_monthly.csv", date_format = "m/d/yyyy")
```

```
## Warning: There were 12 warnings in `mutate()`.  
## The first warning was:  
## i In argument: `Date = case_when(...)`.  
## Caused by warning:  
## ! 361 failed to parse.  
## i Run `dplyr::last_dplyr_warnings()` to see the 11 remaining warnings.
```

```
M48_Industry_Portfolios <- clean_dataframe(X48_Industry_Portfolios, "vw_48_monthly.csv", date_format = "m/d/yyyy")
```

```
## Warning: There were 50 warnings in `mutate()`.  
## The first warning was:  
## i In argument: `Date = case_when(...)`.  
## Caused by warning:  
## ! 361 failed to parse.  
## i Run `dplyr::last_dplyr_warnings()` to see the 49 remaining warnings.
```

```
M100_book_to_market <- clean_dataframe(X100_book_to_market, "vw_100_monthly.csv", date_format = "m/d/yyyy")
```

```
## Warning: There were 2 warnings in `mutate()`.  
## The first warning was:  
## i In argument: `Date = case_when(...)`.  
## Caused by warning:  
## ! 205 failed to parse.  
## i Run `dplyr::last_dplyr_warnings()` to see the 1 remaining warning.
```

```
M25_book_to_market <- clean_dataframe(X25_book_to_market, "vw_25_monthly.csv", date_format = "monthly")
```

```
## Warning: There were 2 warnings in `mutate()`.  
## The first warning was:  
## i In argument: `Date = case_when(...)`.  
## Caused by warning:  
## ! 205 failed to parse.  
## i Run `dplyr::last_dplyr_warnings()` to see the 1 remaining warning.
```

```
# ===== MISSING VALUES CHECK =====
```

```
# Check total number of NA values in the two datasets where we know there are missing values  
sum(is.na(M100_book_to_market)) # Monthly 100 Size-BM
```

```
## [1] 1194
```

```
sum(is.na(D100_book_to_market)) # Daily 100 Size-BM
```

```
## [1] 10485
```

```
# ===== IMPUTATION STRATEGY =====
```

```
# We apply Last Observation Carried Forward (LOCF) and backward fill  
library(zoo)
```

```
## Warning: le package 'zoo' a été compilé avec la version R 4.3.3
```

```
##  
## Attachement du package : 'zoo'
```

```
## Les objets suivants sont masqués depuis 'package:base':  
##  
## as.Date, as.Date.numeric
```

```
M100_book_to_market[,-1] <- na.locf(M100_book_to_market[,-1], na.rm = FALSE)  
M100_book_to_market[,-1] <- na.locf(M100_book_to_market[,-1], fromLast = TRUE, na.rm = FALSE)
```

```
D100_book_to_market[,-1] <- na.locf(D100_book_to_market[,-1], na.rm = FALSE)  
D100_book_to_market[,-1] <- na.locf(D100_book_to_market[,-1], fromLast = TRUE, na.rm = FALSE)
```

```
# Final NA check after imputation  
sum(is.na(M100_book_to_market)) # Should be 0
```

```
## [1] 5
```

```
sum(is.na(D100_book_to_market)) # Should be 0
```

```
## [1] 0
```

```

# na consecutive

which(apply(M100_book_to_market, 1, function(x) any(is.na(x))))

## [1] 4586 4587 4588 4589 4590

M100_book_to_market[which(apply(M100_book_to_market, 1, function(x) any(is.na(x)))), 1:5]

## # A tibble: 5 x 5
##   `SMALL LoBM` `ME1 BM2` `ME1 BM3` `ME1 BM4` `ME1 BM5`
##   <dbl>      <dbl>      <dbl>      <dbl>      <dbl>
## 1      NA    -0.00355    -0.00569    -0.00157    -0.00278
## 2      NA    -0.00359    -0.00588    -0.00167    -0.00273
## 3      NA    -0.00560    -0.00600    -0.00156    -0.00278
## 4      NA    -0.00573    -0.00593    -0.00193    -0.00312
## 5      NA    -0.00594    -0.00608    -0.00191    -0.00318

impute_edges <- function(df) {
  for (j in 2:ncol(df)) { # on ignore la colonne Date
    col <- df[[j]]

    # Complétion au début
    if (is.na(col[1])) {
      first_non_na <- which(!is.na(col))[1]
      if (!is.na(first_non_na)) {
        col[1:(first_non_na - 1)] <- col[first_non_na]
      }
    }

    # Complétion à la fin
    if (is.na(col[length(col)])) {
      last_non_na <- tail(which(!is.na(col)), 1)
      if (!is.na(last_non_na)) {
        col[(last_non_na + 1):length(col)] <- col[last_non_na]
      }
    }

    df[[j]] <- col
  }
  return(df)
}

M100_book_to_market <- impute_edges(M100_book_to_market)
sum(is.na(M100_book_to_market)) # maintenant doit donner 0

## [1] 5

```